CEFET-MG · ALGORITMOS E ESTRUTURAS DE DADOS II · PROF. MICHEL PIRES DA SILVA

Programação Dinâmica

Fundamentos teóricos, modelagem algorítmica e análise experimental

INTEGRANTES DO GRUPO

Alisson Henrique Gomes

Carlos Daniel Barbosa Silveira

João Gabriel Santos Oliveira

João Pedro Ribeiro Santos

Paulo Henrique de Souza Hemetrio

01 · MOTIVAÇÃO

Por que estudar Programação Dinâmica?

$$O(2^n)$$

A armadilha da explosão combinatória

Testar todas as possibilidades às cegas gasta tempo recalculando exatamente os mesmos caminhos.

Escolha imediata falha

Decisões gulosas locais ignoram ganhos futuros.

Retrabalho invisível

Subproblemas idênticos reaparecem milhões de vezes.

A pergunta central

Como lembrar o que já foi resolvido?

O que é Programação Dinâmica?

Método que transforma busca exaustiva em consulta eficiente ao resolver cada subproblema **uma única vez**.



Subestrutura Ótima

É quando a solução ideal de um problema é construída a partir das soluções ideais dos seus subproblemas.



Subproblemas Sobrepostos

Ocorre quando um algoritmo divide um problema e resolve repetidamente os mesmos subproblemas, em vez de recalculá-los.



Reutilização em Memória

Consiste em descartar resultados antigos de subproblemas e armazenar apenas o necessário para o próximo passo.

03 · DESAFIO INTERATIVO

Desafio para a Turma: Corte de Hastes

Temos uma barra de tamanho **n = 4**. Podemos vendê-la inteira ou cortá-la em pedaços inteiros:

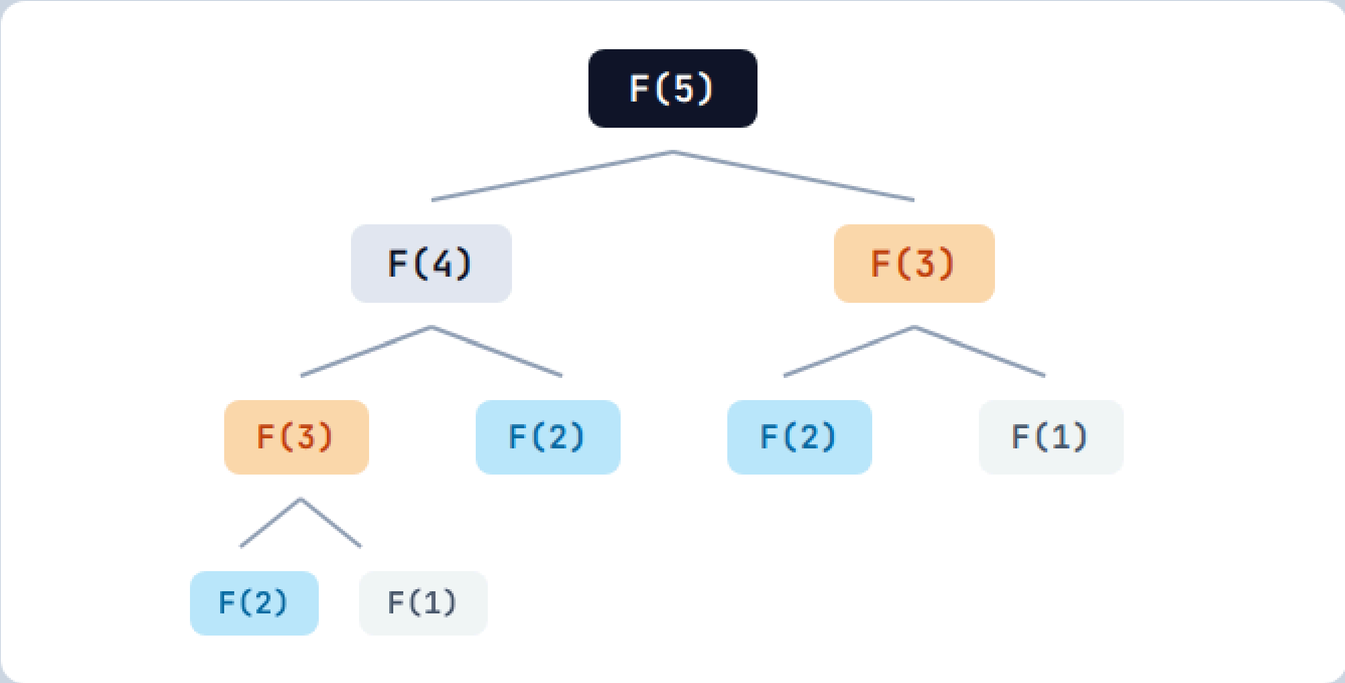
Unidade 1	Unidade 2	Unidade 3	Unidade 4

Comprimento (i)	1	2	3	4
Preço p(i)	R$ 1	R$ 5	R$ 8	R$ 9

Qual é a maneira mais lucrativa de vender essa haste?

04 · SOBREPOSIÇÃO VISUAL

Fibonacci: Árvore com Estados Repetidos



`F(n) = F(n-1) + F(n-2)`

A definição mesmo sendo simples enconde uma árvore de estados repetidos

Subproblemas Recalculados

F(3) abre 2 vezes; **F(2)** abre 3 vezes.

Solução com PD

Podar ramos repetidos usando cache.

Memorização, Tabulação e Espaço O(1)

TOP-DOWN

Memorização

Recursão sob demanda com cache.

`Tempo Θ(n) · Espaço Θ(n)`

BOTTOM-UP

Tabulação

Iterativo dos casos-base ao topo.

`Tempo Θ(n) · Espaço Θ(n)`

OTIMIZAÇÃO

Espaço O(1)

Guarda apenas os dois últimos valores.

`Tempo Θ(n) · Espaço Θ(1)`

Problemas relacionados a cada método:

Memorização: Recursão pode estourar a pilha para valores muito altos.

Tabulação: Gasto excessivo de mémoria armazenando valores que só seram lidos uma vez.

Espaço O(1): Recalcula toda vez que executa.

TABELA DE ESTADOS (F0 ATÉ F6)

Precisamos manter toda a tabela na memória se usamos apenas os 2 anteriores?

0	1	1	2	3	5	8

06 · RESOLUÇÃO DO DESAFIO

Corte de Hastes: Resposta e Recorrência

Nem sempre, a melhor escolha imediata é a melhor:

ESCOLHA IMEDIATA (INTEIRA)

1 pedaço de tamanho 4

R$ 9

SOLUÇÃO ÓTIMA (CORTADA)

2 pedaços de tamanho 2

R$ 10

EQUAÇÃO DE RECORRÊNCIA

$$r(n) = \max \{ p(i) + r(n - i) \}$$

Para $1 \leq i \leq n$, com caso-base $r(0) = 0$.

Os 4 Passos de Cormen na Prática

1. **Estrutura ótima:** Primeiro corte i + ótimo de (n − i).
2. **Recorrência:** Testar todos os cortes i possíveis.
3. **Cálculo de estados:** Resolver r(1), r(2), r(3) até r(n).
4. **Reconstrução:** Guardar o corte ótimo em vetor auxiliar.

A fórmula significa que o lucro máximo de uma haste é a melhor combinação possível entre o preço do primeiro pedaço cortado e o lucro máximo do pedaço restante. O algoritmo testa todos os tamanhos de corte possíveis e escolhe aquele que dá o maior valor financeiro.

07 · DEMONSTRAÇÃO PRINCIPAL

Cadeia de Matrizes: A Ordem Importa?

A = 10 × 30	B = 30 × 5	C = 5 × 60

PARENTIZAÇÃO 1

(A · B) · C

10·30·5 + 10·5·60 = 1.500 + 3.000

4.500 ops

PARENTIZAÇÃO 2

A · (B · C)

30·5·60 + 10·30·60 = 9.000 + 18.000

27.000 ops

Mesmo resultado algébrico, mas **6× menos operações escalares** na ordem ótima. **Economia: 22.500 multiplicações**

Programação dinâmica para encontrar a parentização de custo mínimo

```
1  procedimento MATRIX-CHAIN-ORDER(P[0..N])
2      para i = 1 até n faça
3          m[i,j] = 0
4      para l = 2 até n faça
5          para l = 1 até n-l+1 faça
6              j = i + l - 1
7              m[i,j] = infinito
8              para k = i até j-1 faça
9                  q = m[i,k] + m[k+1,j] + p[i-1]-p[k]-p[j]
10                 se q < m[i,j] então
11                     m[i,j] = q
12                     s[i,j] = k

retornar m, s // m guarda custos mínimos; s guarda o ponto de divisão
```

DIMENSÕES DO EXEMPLO

A = 10 × 30

B = 30 × 5

C = 5 × 60

A tabela m[i, j] armazena o custo mínimo de multiplicar as matrizes de i até j.

A tabela s[i, j] armazena o índice k que define o melhor ponto de divisão para o intervalo de i até j.

Matemática e PD na Cadeia de Matrizes

RECORRÊNCIA DE CUSTO MÍNIMO (PARA $i \le k < j$)

$$m[i,j] = \min \{ m[i,k] + m[k+1,j] + p[i-1] \cdot p[k] \cdot p[j] \}$$

A fórmula funciona dividindo a sequência de matrizes em dois grupos através de um ponto de corte (k). Para cada opção de corte, o algoritmo soma o custo de resolver o lado esquerdo, o lado direito e a multiplicação final entre eles, selecionando no final a posição de corte que gerar o menor gasto total de operações.

$m[i,k] + m[k+1,j]$

Soma dos custos ótimos das subcadeias esquerda e direita.

$p[i-1] \cdot p[k] \cdot p[j]$

Custo de multiplicar as duas matrizes resultantes.

$\Theta(n^3)$ / $\Theta(n^2)$

Preenchimento por diagonais (comprimento crescente).

Demonstração ao vivo no Simulador PD LAB (Instância de Cormen: 15.125 ops)

```
p = [30, 35, 15, 5, 10, 20, 25]
```

Mochila 0/1 e Maior Subsequência Comum (LCS)

Mochila 0/1 `custo:Θ(n · W)`

O problema da mochila 0/1 é um desafio clássico de otimização matemática e computação.

Imagine que você tem uma mochila com uma capacidade de peso limitada e vários itens disponíveis. Cada um desses itens possui um peso e um valor econômico (ou utilidade) diferente.

O grande dilema é: quais itens escolher para colocar na mochila de forma que a soma de seus valores seja a maior possível, sem estourar o limite de peso?

```
dp[i][w] = max(dp[i-1][w], v[i] + dp[i-1]
[w-peso[i]])
```

Estado: (item atual i, capacidade restante w).
Aplicação: Alocação de orçamento e cargas.

LCS (Subsequência) `custo:Θ(m · n)`

O problema da Maior Subsequência Comum (ou Longest Common Subsequence - LCS) é um desafio clássico de comparação de padrões e computação.

Imagine que você tem duas sequências de textos (como "ABCDE" e "XBYDZ") e precisa encontrar o maior elemento que aparece em ambas, mantendo a ordem original das letras, mesmo que elas não estejam juntas.

O grande dilema é: quais elementos selecionar para formar a maior sequência compartilhada possível entre os dois conjuntos, sem alterar a ordem em que eles aparecem?

```
x[i]==y[j] ? 1 + c[i-1,j-1] : max(c[i-1,j],
c[i,j-1])
```

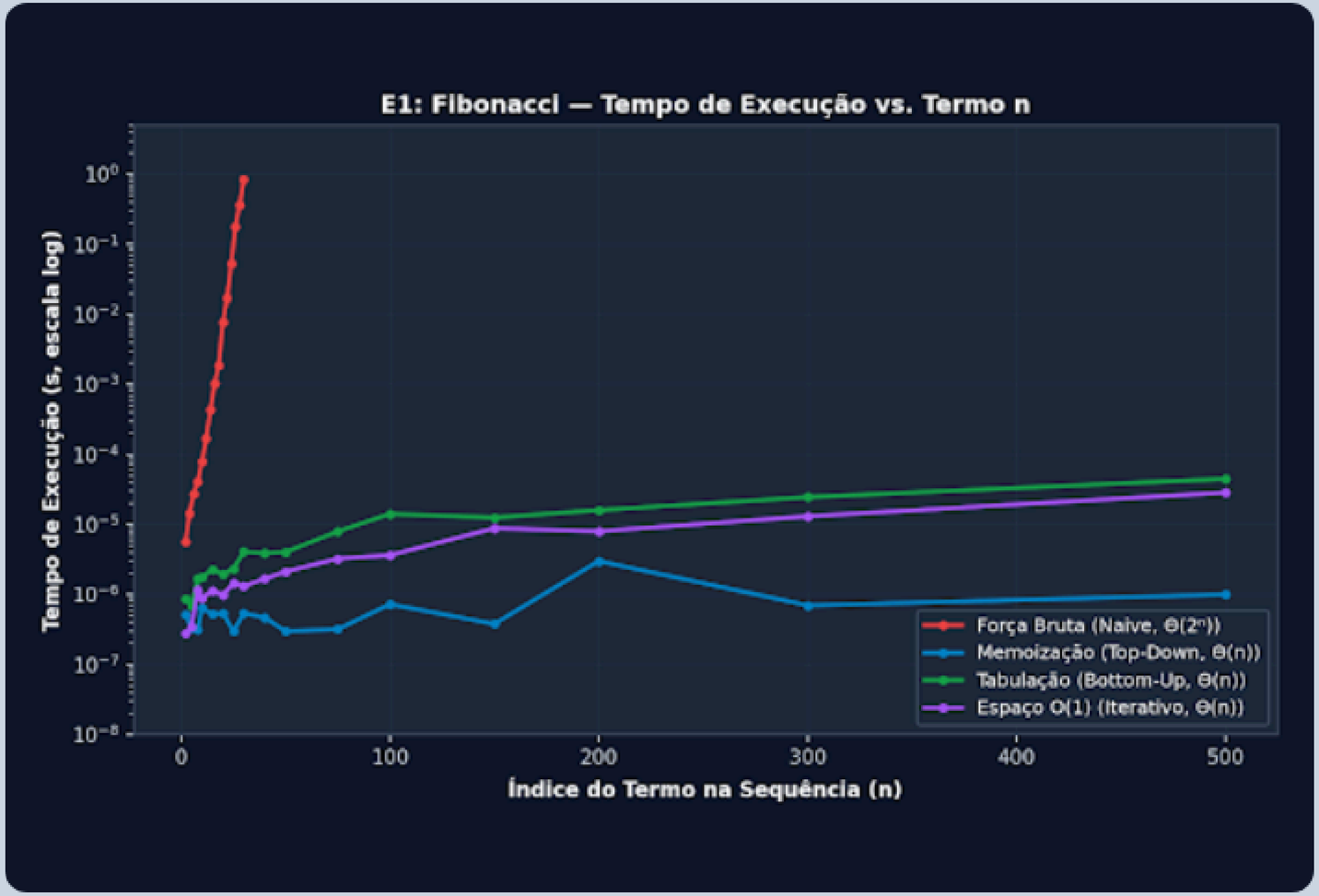
Estado: Prefixos (i, j) das duas strings.
Aplicação: Git diff e alinhamento de DNA.

10 · DIAGNÓSTICO PRÁTICO

Quando Usar a Programação Dinâmica?

Problema	Estado Mínimo	Decisão / Transição	Por que Guloso Falha?	Custo PD
Fibonacci	`Índice n`	Soma dos 2 anteriores	Não é otimização	$\Theta(n)$
Corte de Hastes	`Tamanho n`	Tamanho do 1º corte i	Maior densidade trava resto	$\Theta(n^2)$
Matrizes	`Intervalo (i, j)`	Ponto de divisão k	Menor produto local engana	$\Theta(n^3)$
Mochila 0/1	`Par (i, w)`	Pegar ou deixar item i	Deixa buracos na capacidade	$\Theta(nW)$
LCS	`Prefixos (i, j)`	Casar ou avançar índice	Casamento cedo perde cadeia	$\Theta(mn)$

Experimento 1: Explosão vs. Linearidade (Fibonacci)



FORÇA BRUTA EM N = 30

2.692.537

Chamadas recursivas $\Theta(2^n)$.

MEMOIZAÇÃO EM N = 30

59 chamadas

Queda drástica para 2n − 1.

Conclusão: Eliminar recálculos reduz ordens de grandeza no tempo.

Experimento 2: Otimização de Espaço na LCS

TABELA COMPLETA (N = 1.000)

19,2 MiB

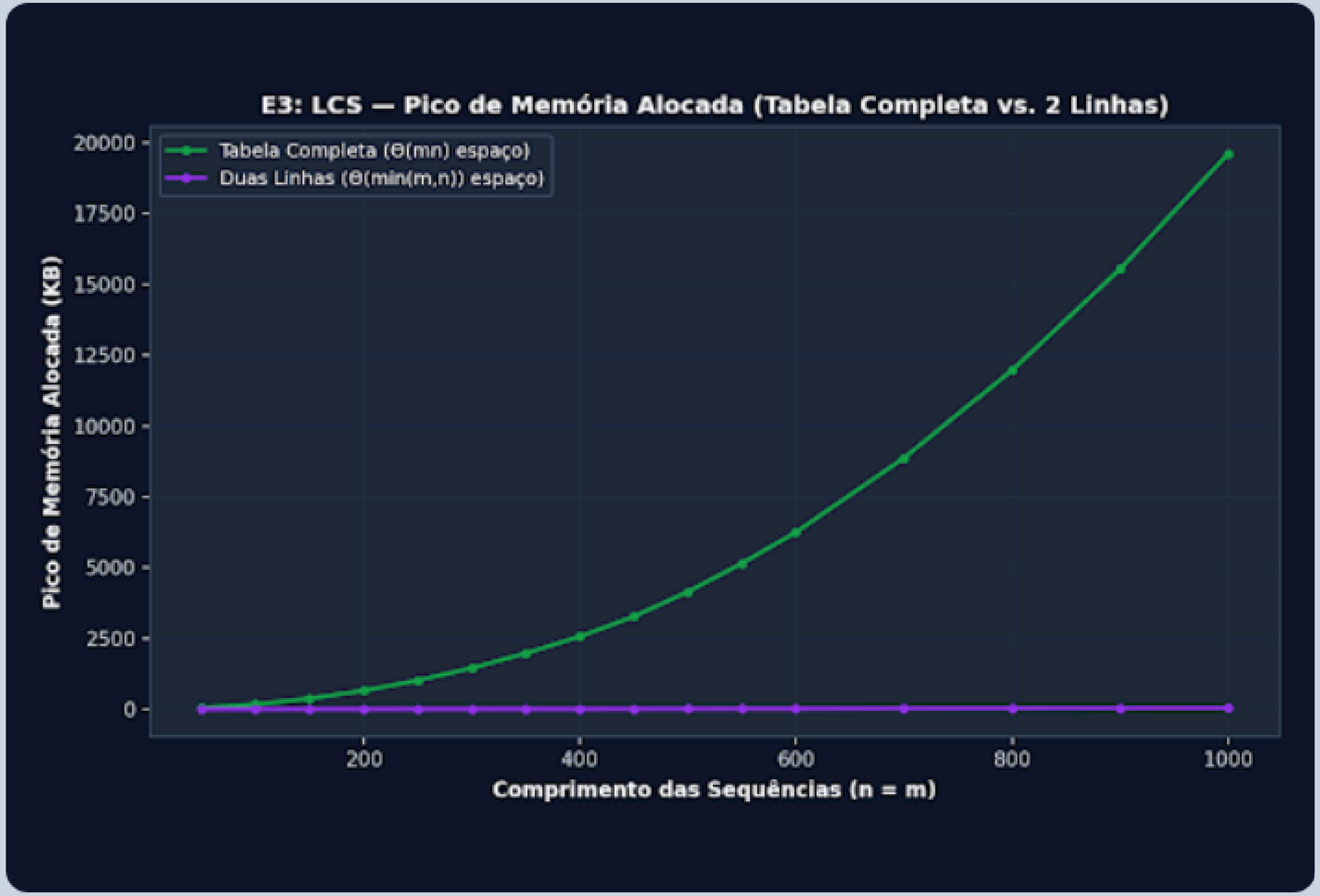
Crescimento quadrático Θ(mn).

DUAS LINHAS (N = 1.000)

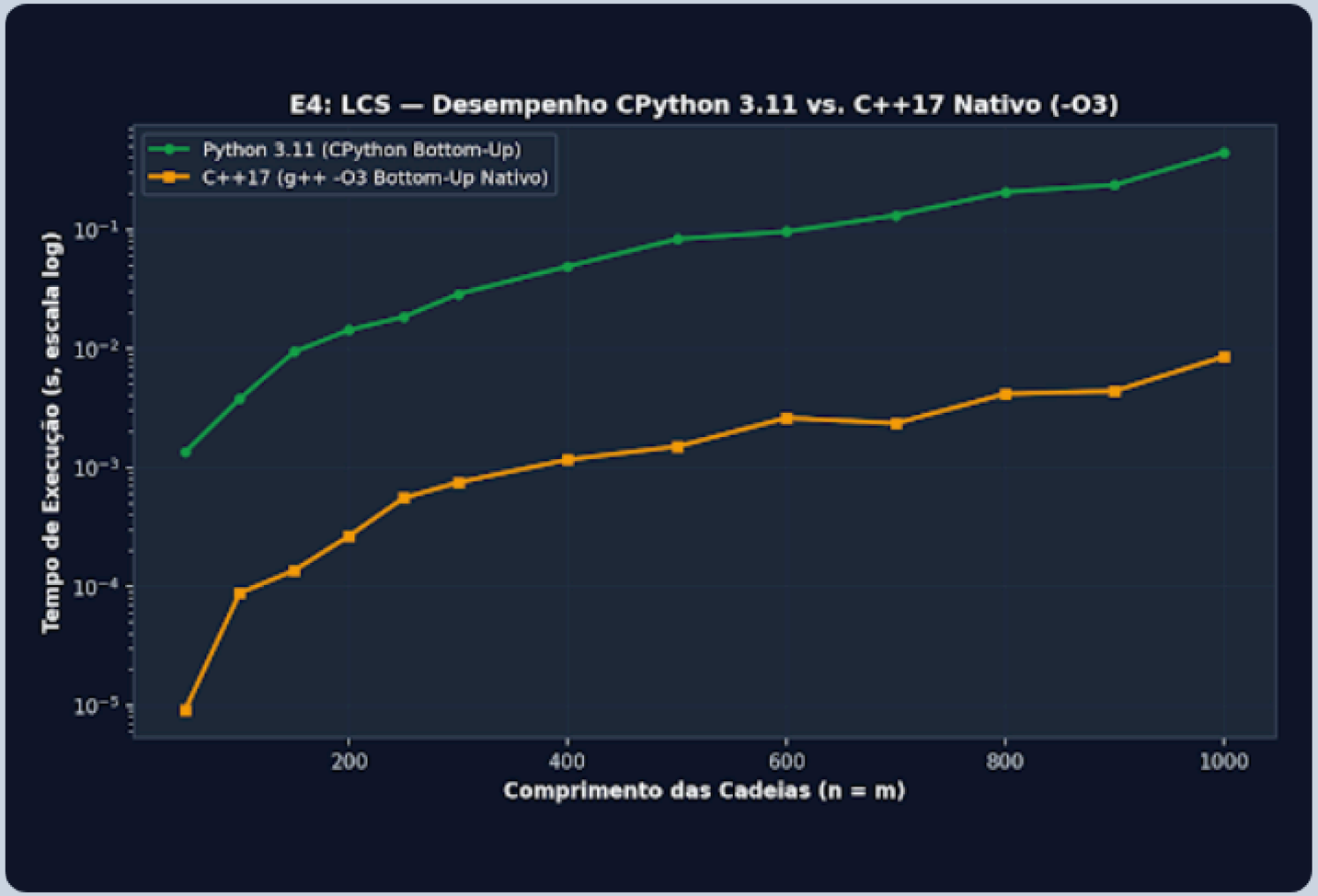
42,9 KiB

Redução de **457×** (−99,8% RAM).

Compensação: Perde reconstrução direta (exige algoritmo de Hirschberg).



Experimento 3: Impacto da Linguagem (Python vs. C++)



SPEEDUP EM N = 1.000

≈ 160×

C++ (1,08 ms) vs. Python (173 ms).

Mesma Complexidade Θ(mn)

Curvas paralelas no gráfico logarítmico provam o mesmo comportamento assintótico.

Importante! A linguagem altera a constante multiplicativa, nunca a classe Big-O.

Limitações da PD Apoiadas pelos Experimentos

Pseudopolinomialidade

Mochila 0/1 custa $\Theta(nW)$: explode quando a capacidade W é muito grande em relação aos bits da entrada.

Limite da Pilha de Recursão

Memoização Top-Down sofre *Stack Overflow* em entradas profundas e adiciona custo de chamadas.

Gargalo de Memória

Tabelas $\Theta(mn)$ ou $\Theta(n^2)$ esgotam a memória RAM rapidamente antes do processador atingir seu limite.

Explosão de Estados (NP-Difíceis)

No Caixeiro-Viajante (Held–Karp), a PD reduz de $O(n!)$ para $O(n^2 \cdot 2^n)$, mas continua exponencial.

15 · SÍNTESE DO SEMINÁRIO

Considerações Finais e Resumo

> “Programação Dinâmica não é calcular mais rápido a mesma coisa. É organizar o problema para nunca recalcular o que já sabemos.”

PILAR 01

Modelagem de Estado

Identificar a informação mínima necessária.

PILAR 02

Transição Segura

Explicitar dependências na recorrência matemática.

PILAR 03

Engenharia de Memória

Equilibrar tempo, espaço e reconstrução.

Entregas do Grupo: 5 famílias de problemas · 8 experimentos (E1–E8) · 48 testes · PD LAB Interativo

Python 3.11 & C++17

ENCERRAMENTO & PRÁTICA INTERATIVA

Obrigado! Perguntas?

Acesse a plataforma interativa desenvolvida pelo grupo para testar o simulador de matrizes e participar do quiz ao vivo:

SIMULADOR DE CADEIA DE MATRIZES

`ribeiro-jp.github.io/Trabalho-AEDS-II-Programacao-Dinamica/matrix-chain.html`

QUIZ INTERATIVO PARA A TURMA

`ribeiro-jp.github.io/Trabalho-AEDS-II-Programacao-Dinamica/quiz.html`



Explore o PD LAB